

Overview and Implementation of the BB84 Quantum Key Distribution Protocol

Uyen Nguyen, Shawn McLain, Ashley Kuta
The Johns Hopkins University, Baltimore, MD, USA
Emails: {unguyen7, smclain4, akuta1}@jh.edu

Abstract—The development of quantum key distribution (QKD) protocols spurs from a need to identify and resolve weaknesses in public key cryptosystems such as RSA. Modern cryptosystems rely on the fact that certain mathematical problems such as factoring are computationally complex on classical computers. Using Shor’s algorithm, a quantum computer can factor a composite positive integer in $\mathcal{O}((\log(N))^3)$ operations. In contrast, a classical computer can only implement sub-exponential time complexity algorithms to solve the factoring problem. Therefore, with the advent of quantum computers, existing public key cryptosystems would have glaring weaknesses against attacks. As a result, seeking alternative protocols that adhere to quantum mechanical principles would provide information-theoretic secure key distribution based on fundamental physics as opposed to classical computer performance [1].

Keywords—BB84, qiskit, IBM, error correction, privacy amplification

I. INTRODUCTION

The foundation of QKD traces back to Stephen Wiesner’s (then unpublished) *Conjugate Coding* manuscript where it was proposed that encoding information in incompatible bases can be used for secure communications. He noted that wave guides could be used as a communication channel to send quantum information encoded in the polarization of light. Messages can be constructed using either the vertical-horizontal or circular polarization basis. Consider the usual computational (Z) basis: $\{|0\rangle, |1\rangle\}$. The two basis states can be represented by photons polarized in the horizontal or vertical direction. These polarizations are orthogonal, so a polarimeter can be oriented such that photons from one of the two states will be detected with certainty. Now, consider the X basis: $\{|-\rangle, |+\rangle\}$. The two basis states can be represented by photons polarized in the left-handed circular or right-handed circular direction. Again, these two polarizations are orthogonal, so measuring in the X basis means a set of photons polarized in this basis can be effectively filtered. However, if the measurement device is set to measure in the X basis while the photons are prepared in the Z basis, or vice-versa, the probability of the measurement device detecting any of the prepared photons is 50 (e.g. $|\langle 0|+\rangle|^2 = 1/2$). Wiesner noted that the two bases are non-commuting observables akin to position and momentum. Only measuring the received information using the correct basis can a deterministic result be discerned. By measuring in the wrong basis, not only is the outcome probabilistic, but uncertainty is introduced because the original state is disturbed by the measurement and no further knowledge about it can be recovered, since identical copies of the unknown, original state can’t be created (in accordance with the no-cloning theorem). Additionally, the fact that a state can’t be recovered makes the

detection of eavesdropping possible by the receiver. Hence, the probabilistic nature of incompatible measurement bases provides a foundation for key distribution protocols because they provide a means of encoding and verifying information. The principles outlined in this manuscript lie at the heart of QKD protocols later developed. [2]

The first QKD protocol was developed in 1984 by Bennett and Brassard. Traditional public key cryptosystems require having a pair of keys, one public key that can be shared over a communication channel, and one private key that must be kept secret for the message to remain secure. The algorithm proposed by Bennett and Brassard creates a shared private key between the sender and receiver, akin to private key cryptosystems. However, unlike private key cryptosystems, the BB84 protocol maintains one of the advantages of public key cryptosystems, which is the distribution of secure random keys over an insecure (quantum) communications channel without the need of a secret key shared a priori, as long as an authenticated classical communication channel is available. [3]

II. STUDY APPROACH

In this section we describe the methodological framework used to study the BB84 quantum key distribution protocol. Our implementation is organized in several logical blocks: (i) the simulation framework and backends, (ii) baseline BB84 state preparation, measurement, and sifting on simulators and hardware, (iii) an intercept-resend eavesdropper model, and (iv) classical post-processing, including error correction and privacy amplification. A Qiskit implementation of this framework can be found at [4].

A. Simulation Framework

We assume that Alice and Bob have already established an authenticated classical communication channel. The BB84 quantum key distribution (QKD) protocol consists of a quantum preparation-measurement phase followed by a classical post-processing phase. In this subsection, we describe how a Qiskit program is constructed to simulate the first phase, combining Alice’s qubit preparation and Bob’s measurements of the transmitted qubits within a single quantum circuit. Eve’s eavesdropping is also incorporated into this circuit.

Our Qiskit program is executed in a hybrid environment: locally and on IBM’s quantum platforms. To run the circuit on IBM hardware, we installed IBM Qiskit Runtime, created an IBM Cloud account, and obtained an access token for the simulation. IBM provides limited free access to its quantum processing units (QPUs), with a cap of 10 minutes of execution

time per month on devices with over 100 qubits. Given these resource constraints, we restrict our implementation of BB84 QKD to 20 qubits for Alice and Bob. We also note that Qiskit is unable to simulate BB84 circuits with 25 or more qubits, due to the exponential growth of the Hilbert space, which reaches dimension 2^N where N is the number of qubits.

While our implementation follows IBM’s BB84 QKD framework [5], we have refactored the code into modular helper functions to improve clarity and reusability. In addition, we also added the key classical post-processing steps: error correction and privacy amplification.

The core components are outlined below:

(a) To implement Alice’s encoding, we define the helper function `encode_bb84_states`, which specifies how Alice prepares her qubits based on her randomly generated bit string and randomly chosen bases. Figure 1 illustrates this publicly known encoding scheme.

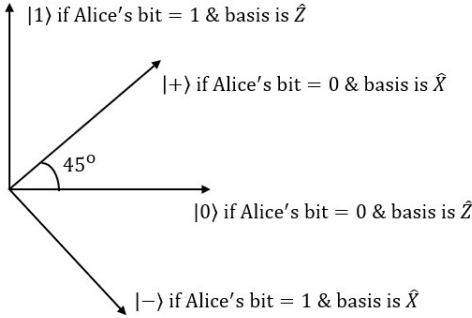


Fig. 1: Bit-to-Qbit encoding based on Alice’s basis

(b) To implement Bob’s measurements of Alice’s qubits, we define two helper functions: `measure_bb84` and `counts_to_bbits`. The first models Bob’s measurement outcomes based on his randomly chosen bases, while the second converts these outcomes into classical bits according to the encoding scheme.

(c) To construct Alice and Bob’s sifted keys, we define the helper function `sift_key`, which compares their basis choices and retains only the bits for which the bases coincide. In practice, Alice and Bob exchange their basis choices over the authenticated classical channel; this exchange is not explicitly simulated here.

(d) To compute the quantum bit error rate (QBER), we define the helper function `key_statistics`, which compares Alice’s and Bob’s sifted keys and calculates the fraction of differing bits. In the ideal case without noise, QBER should be zero.

(e) To simulate the BB84 QKD protocol in the absence of an eavesdropper, we define the helper function `simulate_BB84`. This function accepts a noise model as input, allowing execution either locally or on an IBM quantum platform.

(f) To simulate Eve’s intercept-and-resend attack, we define the helper function `eavesdropper_experiment`. In this routine, Eve intercepts each qubit sent by Alice, measures it using a randomly chosen basis, records the outcome, and forwards the resulting state to Bob.

(g) To correct errors in Bob’s sifted key, we define two key helper functions: `locate_and_fix_error` and `cascade_protocol`. The latter implements the Cascade protocol that corrects errors based on a backtracking procedure on multiple blocks of Bob’s sifted key. The former is used within the latter to find and correct each incorrect bit.

(h) To implement privacy amplification, we define the helper function `generate_toeplitz`, which constructs a random binary Toeplitz matrix. This matrix is used to map the original key to a shorter one, effectively diffusing any information across the resulting key.

B. Baseline BB84 Experiments

The BB84 protocol starts with the sender (Alice) encoding bits in a quantum state (qubits) and transmitting them over a (necessarily insecure) quantum channel. As described above, a physical realization of this is using vertical-horizontal or circular polarizations as a choice of basis each of which each have two orthogonal states in that basis. The receiver (Bob) must choose which basis to measure the incoming qubits without any knowledge of Alice’s choice of basis. Bob has a 50% chance of picking the correct basis, assuming the information is transmitted over a noiseless channel. After every message is sent, Alice and Bob publicly compare their choice of basis chosen for each message, and retain the bits where they both chose the same basis. If the public channel were secure and noiseless, then Alice and Bob would have a shared secret key that can be used for one-time pad encryption.

Figure 2 below shows an implementation of the BB84 protocol as a quantum circuit. The circuit starts with a quantum state in the Z-basis ($\{|0\rangle, |1\rangle\}$), that is then acted upon by an identity gate if Alice chooses to transmit in the z-basis, or is acted upon by a Pauli X gate if Alice chooses to transmit in the x-basis. The meter represents Bob’s readout of the qubit into classical information.



Fig. 2: Communication between Alice and Bob with no eavesdropper.

We implement three baseline scenarios of increasing complexity. Our first baseline corresponds to the ideal case, in which neither noise nor eavesdropping is present in the quantum channel. In this setting, Alice’s state preparation, the quantum channel, and Bob’s measurements are noiseless and free from Eve’s interference. This case is implemented only in local simulation, as IBM hardware is not fully fault-tolerant yet. Under these assumptions, Alice and Bob’s sifted keys should be identical; equivalently, $\text{QBER} = 0$.

The second baseline is also a local simulation without Eve’s presence, but incorporates synthetic noise into the circuit via the `simulate_BB84` helper function. Specifically, we use a single-qubit depolarizing noise model with error probability 0.05. This model captures the effect of noise on Alice’s state preparation and Bob’s measurements, both of which involve single-qubit operations.

The final baseline executes the BB84 simulation on IBM's quantum platforms. Since the specific backend is assigned automatically, we do not control which device is used. The goal in this case is to assess the level of noise in real hardware by comparing its performance against the ideal baseline.

C. BB84 with an Intercept-Resend Eavesdropper

In this section, we describe the security of the BB84 QKD protocol, assuming that Eve executes an intercept-resend attack. In this setting, Eve can listen to the classical communication between Alice and Bob but cannot modify the exchanged information. However, she can intercept the qubits sent by Alice, measure them using her chosen bases, and forward the resulting states to Bob. We then briefly describe our simulations of the BB84 QKD protocol in three different scenarios.

The security of the protocol conceptually relies on the no-cloning theorem, which prohibits the perfect duplication of unknown quantum states. To see why this matters, suppose cloning were possible. Eve could copy each of Alice's transmitted qubits, store these copies in quantum memory, and send identical states to Bob. By observing the public discussion phase, she would learn when Alice's and Bob's bases coincide, allowing her to measure her stored qubits in the correct bases and recover the sifted key without introducing detectable errors. The impossibility of cloning ensures that any such interception necessarily disturbs the quantum states, thereby revealing Eve's presence through an increased quantum bit error rate (QBER). In principle, this is how Alice and Bob can detect eavesdropping.

To obtain a quantitative measure of eavesdropping, the security of BB84 also relies on the fact that non-orthogonal quantum states cannot be perfectly distinguished. When Eve intercepts and measures Alice's qubits, her action is effectively analogous to Bob's measurement process. Because the encoding uses non-orthogonal states, Eve's randomly chosen basis matches Alice's only about 50% of the time. In the remaining cases, her measurements yield incorrect or incompatible results. When she forwards these collapsed states to Bob, his measurements agree with Eve's only about 50% of the time on average. As a result of this compounded mismatch, Alice's and Bob's sifted keys differ about 25% of the time, compared to 0% in the ideal case. Therefore, if Eve intercepts and resends all of Alice's qubits, the QBER can reach up to 25% in the absence of noise, and it increases further as noise is introduced.

What happens if Eve intercepts only a fraction of Alice's qubits? In this case, finding a detection threshold is more subtle. In [6], Shor and Preskill analyzed the security of BB84 and derived a conservative bound for detecting eavesdropping. Using information-theoretic arguments, they established an acceptance threshold of approximately 11% for the QBER. Below this threshold, it is generally impossible to securely generate shared keys and perform error correction. Hence, if $\text{QBER} < 11\%$, Alice and Bob proceed to error correction. Otherwise, they terminate the protocol.

To compute this 11% threshold, Shor and Preskill defined the security key rate $R(Q)$ as $R(Q) = 1 - 2H(Q)$, as a function of the qubit error Q , set $R(Q) = 0$, and solved for Q numerically. Here, $H(Q)$ is the binary Shannon entropy of

Q . It is important to note that this result is obtained under asymptotic assumptions, where the key length is taken to be arbitrarily large for analytical tractability. By contrast, modern information-theoretic analyses of QKD security relax these assumptions and instead consider the finite-key regime, which leads to new thresholds that can tolerate higher levels of QBER. A detailed account of these analyses can be found in [7].

In our simulations of the intercept-and-resend attack, we execute the helper function `eavesdropper_experiment` five times and compute the average QBER. Our simulations comprise three scenarios: (1) noiseless simulation; (2) local simulation with noise; (3) simulation on IBM hardware, which is inherently noisy. We will observe the empirical values of QBER in each case and compares it with the upper 25% threshold discussed above.

Figure 3 below conceptually shows an implementation of the BB84 protocol as a quantum circuit including an eavesdropper using an intercept-and-resend attack scheme. The circuit starts with a quantum state in the Z-basis ($\{|0\rangle, |1\rangle\}$), that is then acted upon by an identity gate if Alice chooses to transmit in the z-basis, or is acted upon by a Pauli X gate if Alice chooses to transmit in the x-basis. Eve's performs a readout measurement, then translates the classical bit into a quantum state that is modified by an identity gate or Pauli X gate depending on the desired basis. Bob then performs a readout measurement of Eve's message.

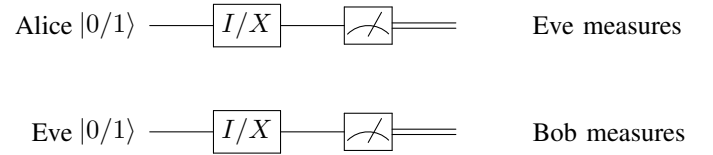


Fig. 3: Intercept-and-resend attack

D. Classical Post-Processing

The QBER can be reduced by classical post-processing step known as information reconciliation, which is an error-correction algorithm performed over the communication channel. It's critical that Bob and Alice perform this correction without leaking too much information to Eve, otherwise the resulting key won't be a secret. There are numerous processes that can be used to find the location of a bit error while minimizing information leakage. One such process is the Cascade protocol. In this protocol, Alice and Bob partition their key strings into blocks of equal size (the block size is inversely proportional to the QBER) and publicly compare the parity of each block. If a block has different parity, then Bob (and Eve) knows that at least one bit flip occurred in his block. Bob and Alice can then perform a binary search on the block by comparing the parity of sub-blocks until a single bit flip is isolated. After an initial passthrough, Alice and Bob permute their key string and repeat the process using a different block size. By doing so, new parity errors in the permuted blocks can be identified, lowering the bit error rate further. Note that this process avoids publicly comparing each bit in the key

over the communication channel, otherwise Eve would have knowledge of the entire key. We implement a simple version of backtracking in which Bob corrects his sifted key to match Alice's sifted key based on the parity of their corresponding bits [8], [9].

Reference [1] provides an information-theoretic justification for privacy amplification. After information reconciliation, Alice and Bob have resolved discrepancies between their respective bit strings creating a shared key, but Eve may have non-negligible information about the key that poses a security risk. Eve's uncertainty about the key is modeled using entropy, more specifically the collision entropy:

$$H_c(X) = -\log\left(\sum_x p(x)^2\right)$$

The objective of privacy amplification is to maximize Eve's entropy. Suppose that Alice and Bob randomly select a hash function, G , from a family of hash functions, and apply it to their respective bit string, W , to yield the m -bit compressed key $G(W) \equiv S$. Given that Eve's known information about the bit string prior to compression is Z , Eve's collision entropy after compression is bounded by:

$$H_c(S|G, Z) \geq m - 2^{m-d} \text{ where } H_c(W|Z) > d$$

Now, suppose that k bits are revealed to Eve via information reconciliation, giving additional knowledge U , in addition to the knowledge gained from eavesdropping prior to the information reconciliation process, Z . Then, Eve's collision entropy of the shared key prior to privacy amplification W is bounded by:

$$H_c(W|Z, U) \geq d - 2k$$

Assuming Eve knows the selected hash function for privacy amplification, G , and has additional knowledge Z and U about the reconciliated bit string, W , applying the randomly selected hash function to the reconciliated bit string leads to the collision entropy of the shared key S being bounded by:

$$H_c(S|G, Z, U) \geq m - 2^{m-(d-2k)}$$

Therefore, to yield an information-theoretic secure key, privacy amplification should yield a m -bit key that satisfies the inequality:

$$m < d - 2k < H_c(W|Z) - 2k$$

In practical implementations, Alice and Bob carry out privacy amplification independently once error correction is complete. We use the Toeplitz matrix hashing method to implement privacy amplification because it is straightforward to generate random binary Toeplitz matrices in Python. Alice and Bob apply the same fixed random binary Toeplitz matrix to their shared key. The multiplication (modulo 2) results in the final shared key. This implicitly assumes that Alice and Bob have agreed on the matrix prior to privacy amplification. This requirement is a limitation because the size of this matrix depends on the length of Alice and Bob's shared key as well as other security parameters, e.g., the estimated bit leakage. See [10], [11] for more discussion on Toeplitz hashing and the process of privacy amplification.

The shared key obtained after privacy amplification can be directly used for one-time pad encryption. Claude Shannon proved that this scheme is information-theoretically secure [12]. That is, its security does not depend on an adversary's computational power, unlike classical or post-quantum cryptographic systems. However, prior to the development of QKD, one-time pad systems were considered impractical because there was no way for two parties to securely generate and share a key without relying on a trusted third party. We illustrate one-time pad encryption by showing how Alice's plaintext "BB84" can be encrypted. To obtain a sufficiently long shared key for this purpose, we first simulate the BB84 protocol under low-noise conditions such that QBER < 11%. Because the number of qubits that can be prepared within a single Qiskit quantum circuit is limited, the simulation must be executed multiple times. We then concatenate the sifted keys obtained from each run. The resulting pair of sifted keys are finally processed through error correction and privacy amplification.

III. RESULTS

A. Simulation of the Quantum Component

This section presents simulation results of the BB84 quantum component with increasing levels of complexity.

1) Local Simulation: Noiseless and without Eavesdropping

The BB84 QKD simulation is executed locally five times using 20 qubits. The outputs presented below correspond to one representative run.

Alice's Bits:	0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1
Alice's Bases:	Z, X, Z, Z, Z, X, X, X, Z, Z, X, Z, X, Z, X, X, Z, X, X, Z, X
Bob's Bases:	X, Z, Z, Z, Z, Z, Z, X, Z, Z, Z, Z, Z, Z, Z, X, Z, X, Z, X
Bob's Bits:	0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1
Alice's Sifted Key	1, 0, 1, 0, 0, 0, 1, 1, 1
Bob's Sifted Key	1, 0, 1, 0, 0, 0, 1, 1, 1

Since Alice and Bob's sifted keys are identical, QBER = 0.

2) Local Simulation: Noisy but without Eavesdropping

We add depolarizing noise to the circuit that simulates Alice's preparations and Bob's measurements. We run this simulation five times and observe the QBER. The outputs presented below are for one representative run.

Alice's Bits:	0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1
Alice's Bases:	Z, X, Z, Z, Z, X, X, Z, Z, X, Z, X, Z, X, Z, X, Z, X, Z, X
Bob's Bases:	X, Z, Z, Z, Z, Z, Z, X, Z, Z, Z, Z, Z, Z, Z, Z, Z, X, Z, X
Bob's Bits:	1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1
Alice's Sifted Key	1, 0, 1, 0, 0, 0, 1, 1, 1
Bob's Sifted Key	1, 1, 1, 0, 0, 0, 1, 0, 1

Since Alice and Bob's sifted keys differ in 2 out of 9 bits, the QBER is $2/9 \approx 22\%$. This increase reflects the added noise. Because this exceeds the 11% threshold, Alice and Bob would abort the protocol in this case.

3) IBM Hardware Simulation: Noisy but without Eavesdropping

This simulation was run on an IBM Quantum Platform repeatedly 10 times. The outputs presented below correspond to one representative run.

Alice's Bits:	0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1
Alice's Bases:	Z, X, Z, Z, Z, X, X, Z, Z, X, Z, X, Z, X, Z, X, Z, X, Z, X
Bob's Bases:	X, Z, Z, Z, Z, Z, Z, X, Z, Z, Z, Z, Z, Z, Z, Z, Z, X, Z, X
Bob's Bits:	0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1
Alice's Sifted Key	1, 0, 1, 0, 0, 0, 1, 1, 1
Bob's Sifted Key	0, 0, 1, 0, 0, 0, 1, 1, 1

We implemented the original BB84 QKD protocol across a variety of scenarios, from the ideal case to more realistic settings with noise and eavesdropping, to highlight the role of quantum information in cryptography, particularly in enabling the one-time pad paradigm. In addition, we incorporated the classical post-processing stages, namely error correction and privacy amplification, to demonstrate the full workflow of the protocol. Our results show that both noise and eavesdropping increase the QBER, thereby affecting Alice and Bob’s decision on whether to abort the protocol or proceed with post-processing. We also discussed the origin of the 11% QBER threshold. Several limitations of our approach were observed. Due to constraints in Qiskit, we were unable to generate sifted keys longer than 25 bits in a single run. Furthermore, our implementation relies on simplifying assumptions made for convenience, although these could readily be relaxed with extra work. Finally, our methods may not be optimal for practical deployment. In particular, more efficient techniques for error correction and privacy amplification are likely available.

REFERENCES

- [1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge: Cambridge University Press, 2010.
- [2] S. Wiesner, "Conjugate coding," *SIGACT News*, vol. 15, no. 1, p. 78–88, Jan. 1983. [Online]. Available: <https://doi.org/10.1145/1008908.1008920>
- [3] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," *Theoretical Computer Science*, vol. 560, pp. 7–11, 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0304397514004241>
- [4] Nguyen, Uyen and McLain, Shawn and Kuta, Ashley, "En.615.781.81.sp26 final project," https://github.com/stm5340/EN.615.781.81.SP26_Final_Project/tree/main, 2026, accessed: 2026-05-02.
- [5] IBM Quantum Platform, "Quantum key distribution," 2017-2026, accessed: 2026-03-29. [Online]. Available: <https://quantum.cloud.ibm.com/learning/en/modules/computer-science/quantum-key-distribution>
- [6] P. W. Shor and J. Preskill, "Simple proof of security of the bb84 quantum key distribution protocol," *Physical Review Letters*, vol. 85, no. 2, p. 441–444, 2000. [Online]. Available: <http://dx.doi.org/10.1103/PhysRevLett.85.441>
- [7] R. Wolf, *Quantum Key Distribution: An Introduction with Exercises*, ser. Lecture Notes in Physics. Cham: Springer, 2021, vol. 988.
- [8] A. Reis, "Quantum key distribution post processing – a study on the information reconciliation cascade protocol," Master's Thesis, Faculdade de Engenharia da Universidade do Porto, Jul. 2019.
- [9] A. Choudhary and A. Wasan, "Cracking the curious case of the cascade protocol," *IEEE Access*, vol. 11, pp. 84 709–84 733, 2023.
- [10] C. H. Bennett, G. Brassard, and J.-M. Robert, "Privacy amplification by public discussion," *SIAM Journal on Computing*, vol. 17, no. 2, pp. 210–229, 1988. [Online]. Available: <https://doi.org/10.1137/0217014>
- [11] G. van Assche, *Quantum Cryptography and Secret-Key Distillation*. Cambridge University Press, Aug. 2010.
- [12] C. E. Shannon, "Communication theory of secrecy systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, Oct. 1949.